Assignment 1

☑ Assignment Concept Summary

Assignment Title: Multi-threaded Client/Server Process Communication using RMI (Remote Method Invocation)
Goal: Demonstrate how a Java program running on one machine can invoke methods on a Java object running on another machine (or JVM), using Java RMI.

🧠 What's Being Done?

This RMI application lets a **client send two numbers to a server**, and the **server returns their sum**. Here's how it's organized:

📁 Code Files

1. **AddServerIntf.java** – Remote interface with the method `add(double, double)`
2. **AddServerImpl.java** – Implements the remote interface on the server
3. **AddServer.java** – Launches the server and binds it to the RMI registry
4. **AddClient.java** – Connects to server and calls `add()` remotely

---

⚙ How to Set Up and Run the Code

You can run this on any machine with **JDK 1.8 or above** installed.

📌 STEP-BY-STEP SETUP

☑ 1. Compile All Java Files

```
javac *.java
```

☑ 2. Generate Stub for RMI

```
rmic AddServerImpl
```

This creates `AddServerImpl_Stub.class` (used by the client to communicate with the server)

☑ 3. Copy Files

Client Folder:

- `AddClient.class`
- `AddServerImpl_Stub.class`
- `AddServerIntf.class`

Server Folder:

- `AddServerIntf.class`
- `AddServerImpl.class`
- `AddServerImpl_Stub.class`
- `AddServer.class`

☑ 4. Start the RMI Registry

Open a terminal and run:

```
rmiregistry
```

Keep this terminal open; it must run in the background.

☑ 5. Start the Server

In a new terminal window:

```
java AddServer
```

☑ 6. Run the Client

In another terminal window:

```
java AddClient 127.0.0.1 4 5
```

Replace `127.0.0.1` with the server IP if running on a different machine.

🖥️ Example Output

```
The first number is: 4
The second number is: 5
The sum is: 9.0
```

📌 Summary

You:

- Compile the files
- Generate RMI stubs
- Start `rmiregistry`
- Run the server
- Run the client to perform remote addition

Great! Let's break down the **code files**, what each one does, and **when/how to run them** in sequence.

☑ Code Explanation and Execution Order

---

1. `AddServerIntf.java`

What it is:
This is the **Remote Interface** – it declares the method(s) that can be called remotely by clients.

```
import java.rmi.*;

public interface AddServerIntf extends Remote {
    double add(double d1, double d2) throws RemoteException;
}
```

Key Point:

- Must extend `java.rmi.Remote`
- All methods must throw `RemoteException`

---

2. `AddServerImpl.java`

What it is:
This is the **implementation** of the above interface. It defines what happens when the `add()` method is called.

```
import java.rmi.*;
import java.rmi.server.*;

public class AddServerImpl extends UnicastRemoteObject implements
AddServerIntf {
    public AddServerImpl() throws RemoteException {}

    public double add(double d1, double d2) throws RemoteException {
        return d1 + d2;
    }
}
```

Key Point:

- Must extend `UnicastRemoteObject`
- Implements the logic to return the sum

---

3. `AddServer.java`

What it is:
This is the **Server Main Class**. It:

- Creates an object of `AddServerImpl`
- Binds it to the RMI registry with the name `"AddServer"`

```
import java.net.*;
import java.rmi.*;

public class AddServer {
    public static void main(String args[]) {
        try {
            AddServerImpl addServerImpl = new AddServerImpl();
            Naming.rebind("AddServer", addServerImpl);
            System.out.println("in server side");
        } catch(Exception e) {
            System.out.println("Exception: " + e);
        }
    }
}
```

---

4. `AddClient.java`

What it is:
This is the **Client Program**. It:

- Looks up the remote object from the RMI registry
- Sends two numbers
- Calls `add()` remotely
- Prints the result

```
import java.rmi.*;

public class AddClient {
    public static void main(String args[]) {
        try {
            String addServerURL = "rmi://" + args[0] + "/AddServer";
            AddServerIntf addServerIntf = (AddServerIntf)
Naming.lookup(addServerURL);

            System.out.println("The first number is: " + args[1]);
            double d1 = Double.valueOf(args[1]);

            System.out.println("The second number is: " + args[2]);
            double d2 = Double.valueOf(args[2]);

            System.out.println("The sum is: " + addServerIntf.add(d1, d2));
        } catch(Exception e) {
            System.out.println("Exception: " + e);
        }
    }
}
```

▶ Sequence to Compile and Run the Code

☑ Step-by-step Execution (on the same machine for simplicity):

🔧 1. Compile All Java Files

```
javac *.java
```

🔧 2. Generate RMI Stub

```
rmic AddServerImpl
```

This generates `AddServerImpl_Stub.class`.

📦 3. Organize Files

You can keep everything in one folder. If running on separate machines:

- Server gets: `AddServer.class`, `AddServerImpl.class`, `AddServerImpl_Stub.class`, `AddServerIntf.class`
- Client gets: `AddClient.class`, `AddServerImpl_Stub.class`, `AddServerIntf.class`

🚀 4. Start RMI Registry (in background)

```
rmiregistry
```

Keep this terminal open. You must run this in the **same folder** as compiled `.class` files.

🚀 5. Run the Server (in a new terminal)

```
java AddServer
```

This will register the `AddServerImpl` object as `"AddServer"` in the registry.

🚀 6. Run the Client (in another new terminal)

```
java AddClient 127.0.0.1 8 9
```

- `127.0.0.1` is localhost (same machine); replace with IP if client is on another computer
- `8` and `9` are numbers to be added

☑ Sample Output

```
The first number is: 8
The second number is: 9
The sum is: 17.0
```

Summary Table

File	Role	Compile?	Run?	When?
`AddServerIntf.java`	Interface declaration	✅	❌	First
`AddServerImpl.java`	Server implementation	✅	❌	After interface
`AddServer.java`	Server main class	✅	✅	After RMI registry
`AddClient.java`	Client main class	✅	✅	After server starts

Assignment 2

Based on your assignment (Assignment No. 2 in the PDF), here's a **complete explanation** of what the assignment is about and **how to run it**, step-by-step.

☑ Concept of the Assignment

You are building a **distributed application** using **CORBA (Common Object Request Broker Architecture)** in Java. The example used is a **Reverse String application**, where:

- The **client sends a string to the server**.
- The **server reverses the string** and sends it back.

This demonstrates **object brokering** across a distributed network, which is CORBA's main feature.

🛠️ Setup & Run Instructions

🔧 Prerequisites

- Java JDK 1.8 installed
- `idlj` compiler (part of JDK for CORBA support)
- Command line terminal (Linux/macOS or Windows CMD/Powershell with Java in PATH)

📁 File Structure

Create a directory `ass3/` and place the following files inside it:

- `ReverseModule.idl`
- `ReverseServer.java`
- `ReverseClient.java`
- `ReverseImpl.java`

🔄 Step-by-Step Execution

1. Generate Java Classes from IDL

```
idlj -fall ReverseModule.idl
```

This generates:

- `Reverse.java`
- `ReverseHelper.java`
- `ReverseHolder.java`
- `ReverseOperations.java`
- `ReversePOA.java`
- Directory: `ReverseModule/`

---

2. Compile All Java Files

```
javac *.java ReverseModule/*.java
```

---

3. Start the ORB Naming Service

```
orbd -ORBInitialPort 1050 &
```

This service is required for the client and server to find each other.

---

4. Run the Server

```
java ReverseServer -ORBInitialPort 1050 -ORBInitialHost localhost
```

You should see:

```
Reverse Object Created
Step1
Step2
Step3
Step4
Reverse Server reading and waiting...
```

---

5. Run the Client

```
java ReverseClient -ORBInitialPort 1050 -ORBInitialHost localhost
```

Then enter any string when prompted. Example:

```
Enter String=
hello corba
Server Send abroc olleh
```

💡 Code Explanation

☑ `ReverseModule.idl`

Defines the interface:

```
module ReverseModule {
  interface Reverse {
    string reverse_string(in string str);
  };
};
```

☑ `ReverseImpl.java`

Implements the reverse logic:

```
public String reverse_string(String name) {
    StringBuffer str = new StringBuffer(name);
    str.reverse();
    return ("Server Send " + str);
}
```

☑ `ReverseServer.java`

- Initializes the CORBA environment
- Registers the `ReverseImpl` object with the naming service

☑ **`ReverseClient.java`**

- Looks up the "Reverse" object
- Sends a string to be reversed
- Receives and prints the reversed result

Assignment 3

This assignment (Assignment No. 3) is about **parallel computing** using the **Message Passing Interface (MPI)** in **Java**, specifically with the **MPJ Express** library. Below is a clear breakdown of what it's about, how to set it up, how to run it, and what the code does:

🔍 Concept Overview

You are building a **distributed system** to calculate the **sum of N elements** using **n processors**. Each processor calculates the sum of a subset (N/n elements), and the results are gathered to compute the final result.

⚙️ Setup and Requirements

🛠️ Prerequisites

- Java JDK 1.8 or higher
- MPJ Express v0.44
    - A Java MPI (Message Passing Interface) library

📦 Installation Steps

1. **Download MPJ Express v0.44** from the official site.
2. **Extract the folder** and locate `mpj.jar` inside the `/lib` directory.
3. **Set environment variables:**

```
export MPJ_HOME=/path/to/mpj
export PATH=$MPJ_HOME/bin:$PATH
```

(You can add these lines to your `~/.bashrc` to make them permanent)

💻 Running the Program

1. **Navigate** to the directory containing `ScatterGather.java`.
2. **Compile the code**:
3. `javac -cp $MPJ_HOME/lib/mpj.jar ScatterGather.java`
4. **Run the code** using `mpjrun.sh` with 4 processes (for example):
5. `$MPJ_HOME/bin/mpjrun.sh -np 4 ScatterGather`
6. **Fixing Permission Issues** (if any):
7. `chmod +x $MPJ_HOME/bin/mpjrun.sh`

📘 Code Explanation: `ScatterGather.java`

```
import mpi.MPI;

public class ScatterGather {
    public static void main(String args[]) {
        MPI.Init(args); // Initialize MPI

        int rank = MPI.COMM_WORLD.Rank();  // Process ID
        int size = MPI.COMM_WORLD.Size();  // Total number of processes
        int root = 0;  // The root process (ID 0)

        int sendbuf[] = null;

        // Only root initializes the data
        if (rank == root) {
            sendbuf = new int[size];
            sendbuf[0] = 10;
            sendbuf[1] = 20;
            sendbuf[2] = 30;
            sendbuf[3] = 40;

            System.out.print("Processor " + rank + " has data: ");
            for (int i = 0; i < size; i++) {
                System.out.print(sendbuf[i] + " ");
            }
            System.out.println();
        }

        int recvbuf[] = new int[1]; // each process receives one element

        // Scatter data from root to all processes
        MPI.COMM_WORLD.Scatter(sendbuf, 0, 1, MPI.INT, recvbuf, 0, 1,
MPI.INT, root);
        System.out.println("Processor " + rank + " has data: " +
recvbuf[0]);

        // Double the received value
        recvbuf[0] *= 2;
        System.out.println("Processor " + rank + " is doubling the data");

        // Gather all modified data back to root
        MPI.COMM_WORLD.Gather(recvbuf, 0, 1, MPI.INT, sendbuf, 0, 1,
MPI.INT, root);

        if (rank == root) {
            System.out.println("Process 0 has data: ");
```

```
            for (int i = 0; i < size; i++) {
                System.out.print(sendbuf[i] + " ");
            }
        }

        MPI.Finalize(); // Finalize MPI
    }
}
```

☑ What It Does Step-by-Step

1. **Root Process (0)** creates an array `[10, 20, 30, 40]`.
2. **Scatter** sends one element to each process.
3. Each process **doubles** its value (e.g., `10 → 20`, `20 → 40`).
4. Values are **gathered back** to the root.
5. **Root** prints the final array: `[20, 40, 60, 80]`.

Assignment 4

Your assignment is about **implementing the Berkeley Algorithm for clock synchronization in Java**. Here's a breakdown to help you understand the **concept**, **how to run the code**, and **what the code does**.

☑ 1. What is this assignment about?

The **Berkeley Algorithm** is a **clock synchronization algorithm** used in **distributed systems** where there is **no accurate time source**. One node is elected as the **master (or time server)**, which:

- Requests the current time from all other nodes (clients),
- Calculates the average time difference,
- Adjusts its own clock and tells each node how to update its time.

🖥️ 2. How to run this Java code

Step-by-step setup:

1. **Install Java (JDK 1.8 or later)**
   - Download from Oracle
   - Set up environment variables if needed (`JAVA_HOME`, `PATH`)
2. **Save the Code**
   - Create a file called `Berkley.java`

    - Copy the full code from your manual (Page 48–50)
3. **Compile the Code**
   Open Command Prompt or Terminal where `Berkley.java` is located and run:
4. `javac Berkley.java`
5. **Run the Program**
   After successful compilation:
6. `java Berkley`

🧠 3. What does the code do?

Classes & Methods:

```
public class Berkley
```

The main class for implementing the algorithm.

Key Methods:

- **`diff(...)`**: Calculates the time difference between master and each client in seconds.
- **`average(...)`**: Computes the average of time differences.
- **`sync(...)`**: Adjusts all clocks (including master and clients) based on average difference.

Flow of Execution:

1. Gets **current time of the server** using `Date date = new Date();`
2. **Asks user** for the number of nodes and their respective time values (HH:MM:SS).
3. Calculates the **difference in seconds** between each client and the server.
4. Calculates the **average time difference**.
5. Adjusts all clocks accordingly and displays the **synchronized time**.

Example:

If the server is at `3:00:00` and two nodes are at `3:05:00` and `2:55:00`:

- Differences: +5 mins, -5 mins = 0 average
- All clocks will be adjusted toward the average (in this simple case, no adjustment needed)

🧪 4. How to test it (Sample Input/Output)

- Program will ask:
- `Enter number of nodes:`
- `> 2`
-
- `Enter time for node 1`
- `Hours:`

- `> 3`
- `Minutes:`
- `> 5`
- `Seconds:`
- `> 0`
-
- `Enter time for node 2`
- `Hours:`
- `> 2`
- `Minutes:`
- `> 55`
- `Seconds:`
- `> 0`
- Output will show:
    - Time differences
    - Average difference
    - Updated time for server and each node

Assignment 5

☑ Assignment Concept Overview

Aim: Implement the **Token Ring-based Mutual Exclusion Algorithm** using Java.

Concept:

- In distributed systems, multiple processes may need exclusive access to a **Critical Section (CS)**.
- **Token Ring Algorithm** ensures **mutual exclusion** by circulating a special message called a *token* around a logical ring of processes.
- A process can **enter the CS only if it has the token**.
- Once done, it passes the token to the next process in the ring.

🧠 What's Happening in the Code?

There are **three programs**:

1. **MutualServer.java** – Acts like a monitor/log receiver.
2. **ClientOne.java** – First client in the ring.
3. **ClientTwo.java** – Second client in the ring.

How They Work Together

- **Server (MutualServer)**: Receives data from clients for logging/display.
- **ClientOne**:
  - Starts with the **token**.
  - Asks if the user wants to send a message.
  - If "Yes", sends the data to the server, then passes the token to ClientTwo.
- **ClientTwo**:
  - Waits for the **token**.
  - Repeats same behavior.
  - Passes the token back to ClientOne.

💻 How to Setup and Run

🛠️ Prerequisites:

- JDK 1.8 or later installed
- Java files saved as:
  - `MutualServer.java`
  - `ClientOne.java`
  - `ClientTwo.java`

🔁 Compilation (Run these in command prompt or terminal):

```
javac MutualServer.java
javac ClientOne.java
javac ClientTwo.java
```

▶️ Execution Order (in 3 separate terminals):

1. **Run Server** (1st Terminal):
2. `java MutualServer`
3. **Run ClientOne** (2nd Terminal):
4. `java ClientOne`
5. **Run ClientTwo** (3rd Terminal):
6. `java ClientTwo`

📜 Code Flow Summary

🧩 MutualServer.java

- Listens on port 7000.
- Whenever a client sends data (a message), it prints it.

🧩 ClientOne.java

- Connects to `MutualServer` at port 7000 and also acts as a mini-server on port 7001 (to allow ClientTwo to connect).
- Starts with token and sends data if the user types "Yes".

- Forwards the token to ClientTwo.

🧩 ClientTwo.java

- Connects to:
  - Server at port 7000 (to send data).
  - ClientOne at port 7001 (to receive/send token).
- Waits for token, acts based on user input, then passes the token back to ClientOne.

☑ Example Flow

1. **ClientOne** starts and has the token.
2. You type "Yes" and send message → printed by `MutualServer`.
3. Token passed to **ClientTwo**.
4. **ClientTwo** gets its turn.
5. This continues in a loop, emulating the **token ring** concept.

Assignment 6

This assignment is about implementing **leader election algorithms**—specifically the **Bully Algorithm** and the **Ring Algorithm**—in Java. These are used in distributed systems to ensure that one process is designated as a coordinator.

🔧 Setup and Execution Instructions

💻 Requirements:

- **Java Development Kit (JDK 1.8)**
- **Eclipse IDE** or any Java IDE
- Or use **Command Line (CMD/Terminal)** with `javac` and `java` commands

---

🗀 How to Run the Code

Option 1: Using Eclipse

1. Open Eclipse.
2. **Create a Java Project**.
3. **Create a Package** in the project.

4. Add two classes:
    - `Ring.java`
    - `Bully.java`
5. Copy the respective code from the PDF into each file.
6. **Run the files individually** by right-clicking > Run As > Java Application.

Option 2: Using Terminal

1. Save each class in separate files:
    - `Ring.java`
    - `Bully.java`
2. Open terminal and navigate to the folder.
3. Compile using:

```
javac Ring.java
javac Bully.java
```

6. Run the code:

```
java Ring
java Bully
```

🧠 What the Code Does (Concept + Code Walkthrough)

🔄 1. Ring Algorithm

Concept: Each process is arranged in a ring. A process starts an election by passing a message around the ring. The process with the highest ID becomes the coordinator.

How it works:

- You input number of processes and assign unique IDs.
- The last one (with the highest ID) is made **inactive** (to simulate failure).
- You choose a process to start the election.
- The message is passed along the ring (skipping inactives), collecting process IDs.
- The highest ID is chosen as the new **coordinator**.

Key Code Points:

- `Rr` class represents each process (`index`, `id`, `state`, `f`)
- `proc[]` array stores all processes.
- `proc[num-1].state = "inactive"` simulates the coordinator failure.
- Election loops through active processes to collect IDs and chooses the maximum.

🥊 2. Bully Algorithm

Concept: Any process can start an election when it suspects the coordinator is down. It sends messages to higher-numbered processes. The highest-numbered process still active becomes the coordinator.

How it works:

- All 5 processes start as active.
- You can:
    1. **Bring up** a process.
    2. **Bring down** a process.
    3. **Send message** from a process to test if the coordinator is alive.
- If coordinator (process 5) is down and a lower-numbered process tries to communicate, it starts an election.
- The highest active process becomes the coordinator.

Key Code Points:

- `state[]` array keeps track of which processes are up.
- `up(int)` simulates recovery and initiates election.
- `down(int)` simulates failure.
- `mess(int)` is where election messages are sent.

☑ Example Output (Bully)

```
Process up = p1 p2 p3 p4 p5
Process 5 is coordinator

1. up a process.
2. down a process
3. send a message
4. exit
```

Assignment 7

Okay, let's break down this assignment on creating and consuming a simple web service using Java.

Concept

The core concept is to understand and implement a basic web service. Web services are a way for different applications to communicate with each other over a network (like the internet). This is done by using open protocols and standards for exchanging information. In simpler terms, it's about making a piece of software on one machine available for use by other machines.

The assignment focuses on:

- **Creating a Web Service:** Building a simple calculator web service that can perform addition operations.

- **Consuming a Web Service:** Developing a client application that can send requests to this calculator web service and get the results.

Tools and Environment

- **Java Programming Environment:** You'll need Java installed on your machine.
- **JDK 8:** The specific version of Java Development Kit (JDK) mentioned is version 8.
- **Netbeans IDE:** This is the Integrated Development Environment (IDE) used for writing and running the Java code. Netbeans helps in organizing your project, writing code, compiling, and debugging.
- **GlassFish Server:** This is the application server used to deploy and run the web service.

How to Run and Setup the Code

The instructions in the document outline the following steps:

1. **Creating the Web Service (CalculatorWSApplication):**
    - Create a new project in Netbeans.
    - Create a package named "org.calculator".
    - Create a class named "CalculatorWS" within this package.
    - Use Netbeans to create a new web service from the "CalculatorWS" class.
    - Netbeans and Glassfish will handle deploying the web service to the server.
2. **Consuming the Web Service (CalculatorClient):**
    - Create a new project for the client application.
    - Create a package named "org.calculator.client".
    - Add the necessary Java classes for the client to interact with the web service. These classes are generated based on the web service's description.
    - Create a servlet and a JSP (JavaServer Pages) page to provide a user interface for the client. This will allow users to input numbers and see the result from the web service.
3. **Running the Application:**
    - In Netbeans, run the client application project.
    - This will start the GlassFish server (if it's not already running), deploy the client application, and you should be able to access it through a web browser.
    - You'll interact with the web page, enter numbers, and the client application will call the web service to perform the addition and display the result.

Code Explanation (with focus on CalculatorWS.java and CalculatorWS_Client_Application.java)

- **CalculatorWS.java (Web Service Code):**
    - This Java class defines the web service.
    - The annotations (like `@WebService` and `@WebMethod`) are important. They tell the Java system that this class is a web service and which methods can be called over the web.
    - The `add` method is the core logic. It takes two integers as input, calculates their sum, and returns the result.
    - When this code is deployed to the GlassFish server, the server makes the `add` method available as a web service.

- **CalculatorWS_Client_Application.java (Client Code):**
    - This Java class is the client application that uses the web service.
    - It contains code to call the `add` method of the web service.
    - It uses classes (like `CalculatorWSService` and `CalculatorWSPort`) that are automatically generated. These classes handle the communication with the web service.
    - The client code gets the input values, calls the `add` method of the web service, and displays the returned result.

Key Points

- **SOAP:** The example in the document uses SOAP, a protocol for exchanging structured information in web services.
- **JAX-WS:** Java provides JAX-WS to create SOAP-based web services.
- **Netbeans and GlassFish:** These tools simplify the process of creating, deploying, and running web services.